
TRAVAIL DE FIN D'ÉTUDES

Snow Wonder
FESTIVAL



*Application web événementielle single-page
pour un festival hivernal de sculptures de neige*

Claire Manon Siboni

Promoteur: Guillaume Duribreux



TRAVAIL DE FIN D'ÉTUDES

Snow Wonder
FESTIVAL

*Application web événementielle single-page
pour un festival hivernal de sculptures de neige*

Claire Manon Siboni

Promoteur: Guillaume Duribreux

Remerciements

Je souhaite adresser mes premiers remerciements à mon promoteur, Monsieur Guillaume Duribreux, pour sa disponibilité, ses retours constructifs et la confiance qu'il m'a accordée tout au long de ce projet.

Je remercie également Martial Parfait et François Racquez pour la qualité de leurs enseignements et pour avoir su transmettre leur passion du développement web avec pédagogie et bienveillance.

Une pensée particulière va à mes camarades Viktor, Guillaume et Killian, comme moi derniers survivants après deux longues années de formation.

Une mention spéciale à mes sœurs, Corentine et Annelise, testeuses officielles de tous mes projets, qui ont su m'aiguiller par leurs précieux conseils.

Enfin, je remercie du fond du cœur ma famille, mes amis et mes chats, dont le soutien quotidien a été une source d'énergie précieuse tout au long de cette aventure.

Table des matières

Introduction.....	1
Contexte et objectifs.....	1
Méthodes et technologies développées.....	1
Chapitre 1 — Analyse des besoins et planification.....	3
1.1 Analyse des besoins (Impact Mapping).....	3
1.2 User Stories.....	3
1.3 Planning d'exécution.....	4
Chapitre 2 — Conception UI/UX et charte graphique.....	6
2.1 Identité visuelle et charte graphique.....	6
2.2 Inspirations design.....	6
2.3 Wireframes et conception de l'interface.....	6
Chapitre 3 — Architecture technique et choix des outils.....	10
3.1 React et Vite : l'architecture composant.....	10
3.2 TailwindCSS v4 : le design utilitaire.....	10
3.3 Supabase : le Backend-as-a-Service.....	10
3.4 Leaflet.js et Mapbox : la carte interactive.....	10
Chapitre 4 — Analyse de la base de données.....	12
4.1 Structure des tables.....	12
4.2 Relations entre les tables.....	13
Chapitre 5 — Fonctionnalités clés du site.....	14
5.1 Le compte à rebours dynamique (Hero).....	14
5.2 Gestion dynamique des éditions.....	14
5.3 Les formulaires : inscription et billetterie.....	15
5.4 La galerie photo dynamique.....	16
5.5 Le système de scan à l'entrée (TicketScanner).....	17
5.6 Le panneau d'administration (Back-Office).....	18
5.7 Sécurité et protection des données.....	19
Chapitre 6 — Déploiement et mise en production.....	21
6.1 Déploiement sur Vercel.....	21
6.2 Supabase Edge Functions.....	21
6.3 Optimisation des performances.....	21
Chapitre 7 — RGPD et cookies.....	23
7.1 RGPD et protection des données personnelles.....	23
7.2 Gestion des cookies.....	23
Chapitre 8 — Estimation budgétaire.....	24
8.1 Hypothèses de calcul.....	24
8.2 Phases de développement.....	24
8.3 Frais annexes (première année).....	24
8.4 Maintenance annuelle.....	24

8.5 Coût total première année.....	25
Conclusion.....	26
Intégration des savoirs et savoir-faire.....	26
Adéquation entre objectifs et résultats.....	26
Difficultés rencontrées et solutions apportées.....	26
Réflexion personnelle.....	26
Bibliographie.....	28
Ouvrages et règlements.....	28
Documentations techniques.....	28
Sources économiques.....	28
Crédits photographiques.....	28
Intelligence artificielle.....	29
Table des figures.....	30
Annexes.....	31
Annexe A — Schéma du chemin des données.....	31

Introduction

L'idée est née en novembre 2025, pendant une de ces chutes de neige comme on n'en voit plus souvent, le genre qui transforme un trajet en voiture en contemplation forcée et les idées en projets. Snow Wonder Festival s'est imposé à moi comme une évidence : un festival hivernal de sculptures de neige, trois éditions, une application qui ferait tout : billets, inscriptions, scan à l'entrée, contenu modifiable sans toucher au code. L'ironie, c'est que je rends ce travail en pleine canicule de mai, les volets fermés, à des kilomètres du moindre flocon.

Le brief initial était simple : un site événementiel single-page avec carte et système d'inscription. Mais j'avais vécu l'autre côté du miroir — celui de l'organisateur qui dépend d'un développeur pour changer une date, corriger une typo ou ajouter une photo, et c'est épuisant. Alors j'ai revu mes ambitions à la hausse : plusieurs éditions gérables en autonomie, des billets PDF individuels avec QR code, un système de scan à l'entrée en temps réel, et une confirmation e-mail automatique à chaque étape. Tout ça sans redéploiement.

J'avais commencé en HTML/CSS/JS vanilla, le temps de m'approprier le visuel et poser les bases. Mais dès les premiers jours de mon stage, React et TailwindCSS s'étaient imposés comme une évidence — c'est ce que j'utilisais au quotidien, c'est ce qui correspondait à la nature de mon projet. Reprendre une base existante en cours de route, c'était risqué. C'était aussi la décision la plus honnête vis-à-vis du projet.

Contexte et objectifs

Le brief client était assez libre : créer une expérience en ligne mémorable pour un événement, avec système d'inscription intégré. Ce que j'en ai fait est une autre histoire.

Quatre exigences ont structuré l'ensemble du projet :

- ❖ Une application single-page avec un front-office public et un back-office d'administration sécurisé ;
- ❖ Une carte interactive géolocalisée, parce qu'un festival ça se situe quelque part ;
- ❖ Un système de réservation complet ;
- ❖ Une sécurité des données sérieuse, pas juste une case cochée, mais une réflexion à chaque couche de l'application.

Ce n'est pas le brief qui a généré ces exigences. C'est la conviction que si on construit quelque chose, autant le construire bien.

Méthodes et technologies développées

Chaque outil a été choisi pour une raison précise, pas par défaut.

React 19 comme bibliothèque d'interface : pour la gestion réactive des composants, l'architecture modulaire, et parce que c'est ce que le marché utilise.

Vite comme outil de build : démarrage instantané, Hot Module Replacement ultra-rapide, tree-shaking automatique en production.

TailwindCSS v4 pour le design utilitaire et le responsive — styliser directement dans le JSX, sans jongler entre dix fichiers CSS.

Pour le back-end, j'ai choisi **Supabase** plutôt qu'un serveur PHP maison : base de données PostgreSQL, authentification JWT, politiques de sécurité granulaires au niveau de la base, et fonctions serverless en Deno/TypeScript pour les opérations sensibles comme la génération de PDF et l'envoi d'e-mails.

Leaflet.js couplé à **Mapbox** pour la carte interactive — gratuit, open-source, et personnalisable à souhait.

Et **Vercel** pour le déploiement continu, parce qu'un [git push](#) qui met le site en ligne en 30 secondes, ça change la vie.

Ce document présente successivement l'analyse menée en amont, les choix de conception, la mise en œuvre technique, puis une évaluation personnelle du travail accompli.

Chapitre 1 — Analyse des besoins et planification

1.1 Analyse des besoins (Impact Mapping)

Avant d'écrire la moindre ligne de code, j'ai procédé à une analyse rigoureuse des besoins à partir du brief client. Cette analyse s'est articulée autour de quatre catégories d'acteurs (ou impacts) :

- ❖ Les visiteurs : ils ont besoin d'informations claires sur le festival (programme, dates, lieu), d'un accès simple à la billetterie et d'une expérience visuelle immersive qui donne envie de venir.
- ❖ Les participants au concours : ils doivent pouvoir s'inscrire facilement à la compétition (catégorie amateur ou professionnelle), choisir leur(s) édition(s) et recevoir une confirmation.
- ❖ L'organisateur (l'administrateur) : il a besoin d'un back-office complet pour consulter les inscriptions et les commandes de billets, gérer le contenu du site (textes, galerie photo, gagnants, éditions), sans dépendre d'un développeur pour chaque modification.
- ❖ Le système : il doit garantir la sécurité des données, l'envoi automatique des e-mails de confirmation et la disponibilité du site.

De cette analyse sont ressortis les besoins fonctionnels suivants :

Fonctionnalité	Priorité
Présentation du festival (hero, programme, galerie)	Haute
Carte interactive géolocalisée	Haute
Formulaire d'inscription au concours	Haute
Réservation de billets avec confirmation par e-mail	Haute
Interface d'administration sécurisée	Haute
Gestion dynamique des éditions depuis le back-office	Haute
Système de scan QR à l'entrée du festival	Moyenne
Gestion de la galerie photo depuis l'admin	Moyenne
Section palmarès / gagnants	Basse
Newsletter	Basse

1.2 User Stories

L'analyse des besoins s'est structurée autour de quatre périmètres distincts correspondant aux acteurs identifiés dans l'impact mapping.

Périmètre visiteur (front-office public)

- ❖ Consulter le programme des trois éditions du festival à venir (décembre 2026, janvier 2027, février 2027)
- ❖ Naviguer entre les sections via le menu fixe avec ancrs (#schedule, #gallery, #market, #forms, #map, #winners)
- ❖ Visualiser la galerie photo avec lightbox

- ❖ Localiser le site du festival et ses zones sur la carte interactive
- ❖ Consulter le palmarès des éditions passées
- ❖ Accéder au formulaire de contact via le pied de page

Périmètre participant au concours

- ❖ S'inscrire au concours de sculptures de neige en choisissant sa catégorie (amateur ou professionnel) et son/ses édition(s)
- ❖ Être informé en temps réel si son adresse e-mail est déjà inscrite à une édition
- ❖ Recevoir un e-mail de confirmation HTML personnalisé après soumission

Périmètre visiteur souhaitant assister au festival

- ❖ Réserver jusqu'à 10 billets gratuits pour l'édition de son choix
- ❖ Fournir ses coordonnées complètes (nom, email, téléphone, adresse postale)
- ❖ Opter pour l'abonnement à la newsletter
- ❖ Recevoir ses billets PDF individuels par e-mail, chacun portant un QR code unique au format SWF-XXX-XXXXXX

Périmètre administrateur (back-office)

- ❖ Se connecter de manière sécurisée via e-mail et mot de passe avec double vérification (Supabase Auth + table admin_users)
- ❖ Réinitialiser son mot de passe via un lien sécurisé reçu par e-mail
- ❖ Consulter le tableau de bord avec les statistiques clés, les actions rapides et le journal d'activité récente
- ❖ Créer, modifier, activer/désactiver et supprimer des éditions sans redéploiement
- ❖ Filtrer, consulter et supprimer des inscriptions au concours
- ❖ Gérer les commandes de billets et les billets individuels générés
- ❖ Ajouter, supprimer, réorganiser et activer/désactiver les images de la galerie
- ❖ Modifier les textes éditoriaux du site sans redéploiement
- ❖ Saisir et mettre à jour le palmarès des éditions

Périmètre agent de contrôle (scanner)

- ❖ Accéder à une interface épurée limitée au seul module de scan QR code
- ❖ Scanner un billet ou une inscription pour en valider l'entrée en temps réel
- ❖ Recevoir un retour visuel immédiat (vert / orange / rouge) selon le statut du document scanné

1.3 Planning d'exécution

J'ai structuré le projet en quatre grandes phases :

- ❖ Phase 1 — Conception et premiers développements (décembre – janvier) : élaboration de la charte graphique, création du moodboard, réalisation des wireframes, conception du schéma de base de données. Une première version du site a été développée en HTML vanilla, CSS et JavaScript natif, posant les bases de l'architecture et de l'identité visuelle.
- ❖ Phase 2 — Refonte technologique et développement front-end (février – avril) : fort de mon expérience en stage et souhaitant adopter des technologies mieux adaptées à une SPA, j'ai migré le projet vers React et

TailwindCSS. Cette phase a également inclus la configuration de Supabase (tables, politiques RLS, authentification) et le développement des composants principaux de la page publique, des formulaires et de l'intégration de la carte Leaflet.

- ❖ Phase 3 — Back-end, déploiement et mise en ligne (fin avril – début mai) : développement du panneau d'administration, écriture des Edge Functions (envoi d'e-mails, génération de PDF), module de gestion des éditions, système de scan à l'entrée. Le site a été déployé sur Vercel et le domaine snow-wonder.be configuré.
- ❖ Phase 4 — Tests, optimisation et finitions (mai) : tests fonctionnels multi-navigateurs, optimisation des performances (audit Lighthouse, code splitting, conversion WebP), vérification de la sécurité, ajustements visuels et ajout des dernières fonctionnalités.

Chapitre 2 — Conception UI/UX et charte graphique

2.1 Identité visuelle et charte graphique

L'identité visuelle de Snow Wonder Festival ne cherche pas à être moderne au sens où on l'entend aujourd'hui — épurée, froide, typographiquement agressive. Elle cherche autre chose : ce sentiment particulier de regarder la neige tomber par la fenêtre alors qu'on est bien au chaud, un chocolat entre les mains. *Whimsical*, féérique, un peu hors du temps.

La palette chromatique est construite autour de cette sensation :

- ❖ **deep-navy** (#002442) — un bleu nuit profond qui évoque le ciel d'hiver à 17h, cette obscurité douce qui n'a rien d'hostile. La couleur de fond principale.
- ❖ **midblue** (#3A7CA5) — le bleu intermédiaire des dégradés et de la navigation, couche qui donne de la profondeur sans contraste brutal.
- ❖ **ice-blue** (#CAE9FF) — la teinte exacte d'un carreau de fenêtre givré. Textes secondaires et détails décoratifs.
- ❖ **festival-yellow** (#E8A94E) — jaune doré chaleureux, la lumière dans tout ça : la lampe allumée, le feu dans la cheminée. Le point d'accentuation qui réchauffe l'ensemble.

Le fond de page est un dégradé vertical de l'ice-blue vers le midblue puis le deep-navy — une profondeur atmosphérique qui imite naturellement un ciel d'hiver du crépuscule vers la nuit.

Côté typographie, **DM Serif Display** pour les titres apporte le côté éditorial et légèrement romanesque, **Nunito** pour le corps de texte garde une rondeur douce qui colle au registre, et **Rubik** est réservé à l'administration — fonctionnel, neutre, hors de l'univers visuel du festival.

2.2 Inspirations design

La conception visuelle s'est nourrie d'une veille graphique approfondie. Plusieurs sites ont servi de références : And Apartments pour l'élégance des typographies, Move to Be More pour la fluidité du scroll, Tokyo Design Forum pour l'équilibre entre densité et respiration visuelle, Two Hyde Design pour les effets de superposition, Brewitty pour la cohérence de l'identité, Open Lines pour les micro-interactions sur fond sombre, Benjamin Wilkes pour les animations au défilement, Frank Firewood pour l'atmosphère hivernale, et Shirtique Krazykaps pour la mise en valeur photographique.

2.3 Wireframes et conception de l'interface

Les wireframes ont été conçus en respectant les principes fondamentaux de l'UX : hiérarchie visuelle claire, parcours utilisateur intuitif et appels à l'action bien positionnés. La page se décompose en sections empilées verticalement : navigation fixe, Hero avec compte à rebours, Programme, Galerie, Marché, Formulaire, Carte, Gagnants et Pied de page (voir Figure 1).

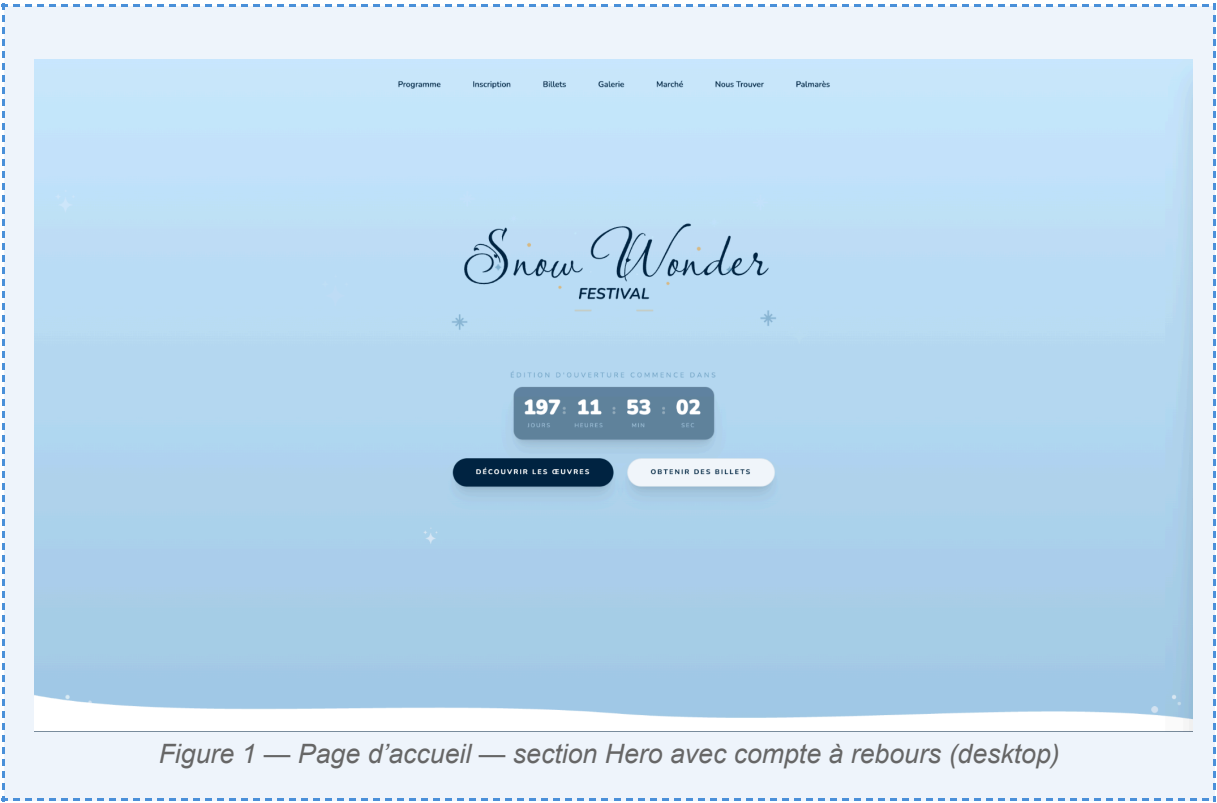
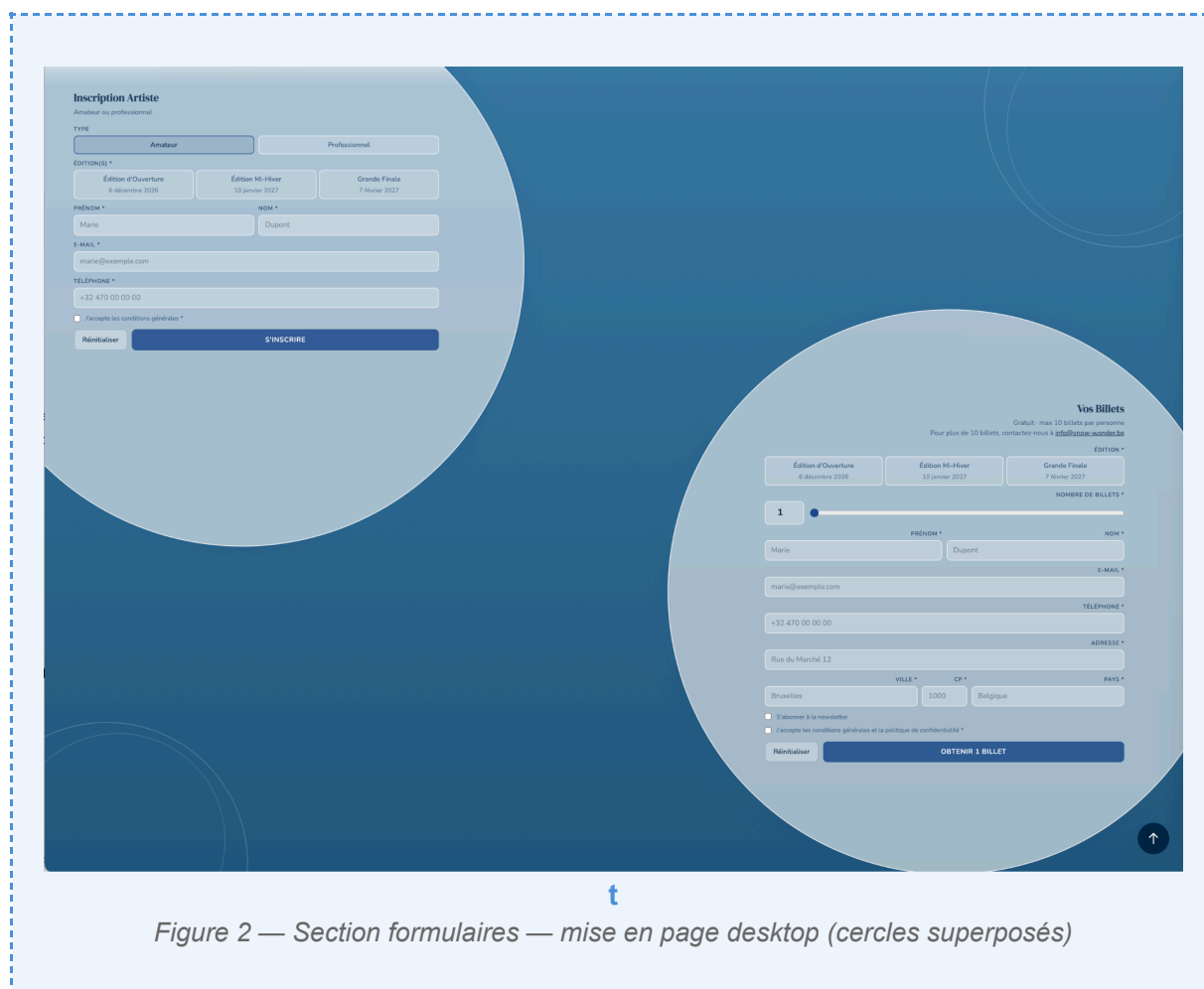


Figure 1 — Page d'accueil — section Hero avec compte à rebours (desktop)

Conception responsive : trois breakpoints ont été définis. Mobile (< 768px) : navigation en menu latéral, formulaires empilés verticalement avec des formes en quart de cercle, galerie en grille 2 colonnes. Tablette (768px–1280px) : formulaires côte à côte dans des cards glassmorphismes. Desktop (> 1280px) : formulaires imbriqués dans deux grands cercles semi-transparents qui se chevauchent (voir Figures 2 et 3).



Vos Billets

Gratuit · max 10 billets par personne
Pour plus de 10 billets, contactez-nous à info@snow-wonder.be

ÉDITION *

Édition d'Ouverture 6 décembre 2026	Édition Mi-Hiver 10 janvier 2027	Grande Finale 7 février 2027
--	-------------------------------------	---------------------------------

NOMBRE DE BILLETS *

1

PRÉNOM * NOM *

Marie Dupont

E-MAIL *

marie@exemple.com

TÉLÉPHONE *

+32 470 00 00 00

ADRESSE *

Rue du Marché 12

VILLE * CP * PAYS *

Bruxelles 1000 Belgique

☐ S'abonner à la newsletter

☐ J'accepte les conditions générales et la politique de confidentialité *

Réinitialiser OBTENIR 1 BILLET

Figure 3 — Section formulaires — mise en page mobile (quart de cercle)

Chapitre 3 — Architecture technique et choix des outils

3.1 React et Vite : l'architecture composant

React s'est imposé pour une raison simple : une SPA de cette envergure, avec un front-office public, un back-office, un scanner et une gestion dynamique des éditions, ne se pilote pas proprement en vanilla selon moi. L'interface est découpée en composants autonomes — `Hero`, `ScheduleSection`, `GallerySection`, `MarketSection`, `FormsSection`, `MapSection`, `WinnersSection` — orchestrés par le composant racine `App.jsx`, et organisés en trois sous-dossiers : `buildingBlocks/`, `modals/` et `sections/`.

Vite plutôt que Create React App : démarrage instantané via les modules ES natifs, Hot Module Replacement immédiat, et un build de production avec tree-shaking et code-splitting automatiques. Une fois qu'on y a goûté, il est difficile de revenir en arrière.

3.2 TailwindCSS v4 : le design utilitaire

TailwindCSS permet de styliser directement dans le JSX, sans jongler entre des fichiers CSS dispersés. Les couleurs de la charte graphique (`bg-deep-navy`, `text-festival-yellow` et consorts) sont définies une fois dans la configuration et disponibles partout. Pour les cas que Tailwind ne couvre pas proprement, comme les cartes de `ScheduleSection` ou les effets glassmorphiques de `FormsSection`, des styles CSS dédiés prennent le relais.

3.3 Supabase : le Backend-as-a-Service

Supabase fournit en un seul service ce qu'il aurait fallu assembler à la main avec un back-end PHP : base de données PostgreSQL, API REST auto-générée, authentification JWT et fonctions serverless. Le Row Level Security granulaire est configuré directement au niveau PostgreSQL — pas dans le code applicatif. La clé anonyme publique (`VITE_SUPABASE_ANON_KEY`) ne permet que les opérations explicitement autorisées par ces politiques ; tout le reste passe par les Edge Functions Deno/TypeScript côté serveur.

3.4 Leaflet.js et Mapbox : la carte interactive

Leaflet.js plutôt que Google Maps : gratuit, compatible OpenStreetMap, et personnalisable via des tuiles Mapbox sans dépendre d'une API payante au premier dépassement de quota.

Le site intègre deux instances distinctes. La première est une carte principale affichant un unique marqueur positionné aux coordonnées GPS précises du festival, avec une icône personnalisée en `L.divIcon` et un popup avec lien Google Maps Directions (voir Figure 4). La seconde est une carte du marché, accessible via une modal dédiée, qui affiche neuf marqueurs : huit stands gastronomiques et un marqueur central identifiant le gazebo (voir Figure 5).

Les deux cartes sont montées via `useRef` et `useEffect`, et nettoyées au démontage pour éviter les fuites mémoire — un détail technique que Leaflet,

bibliothèque DOM impérative, exige qu'on gère explicitement dans un environnement React.

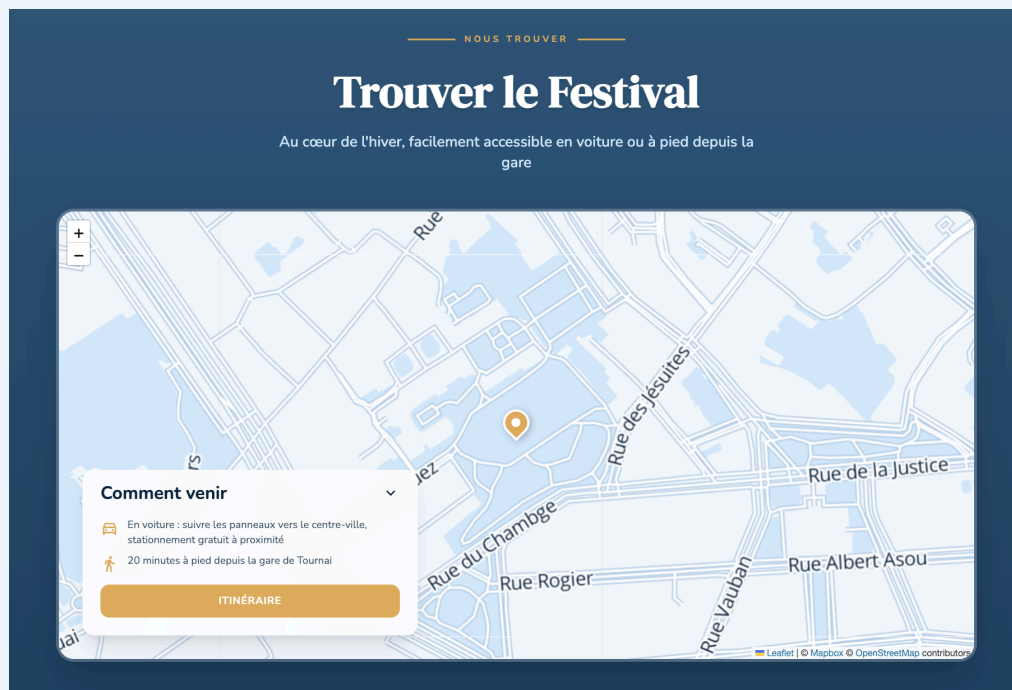


Figure 4 — Carte principale Leaflet — marqueur du site du festival

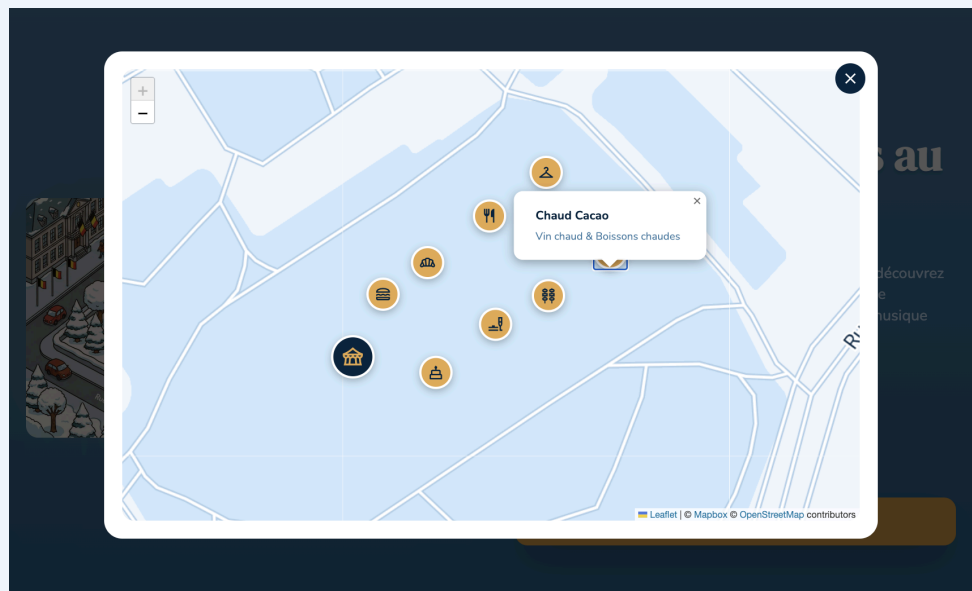
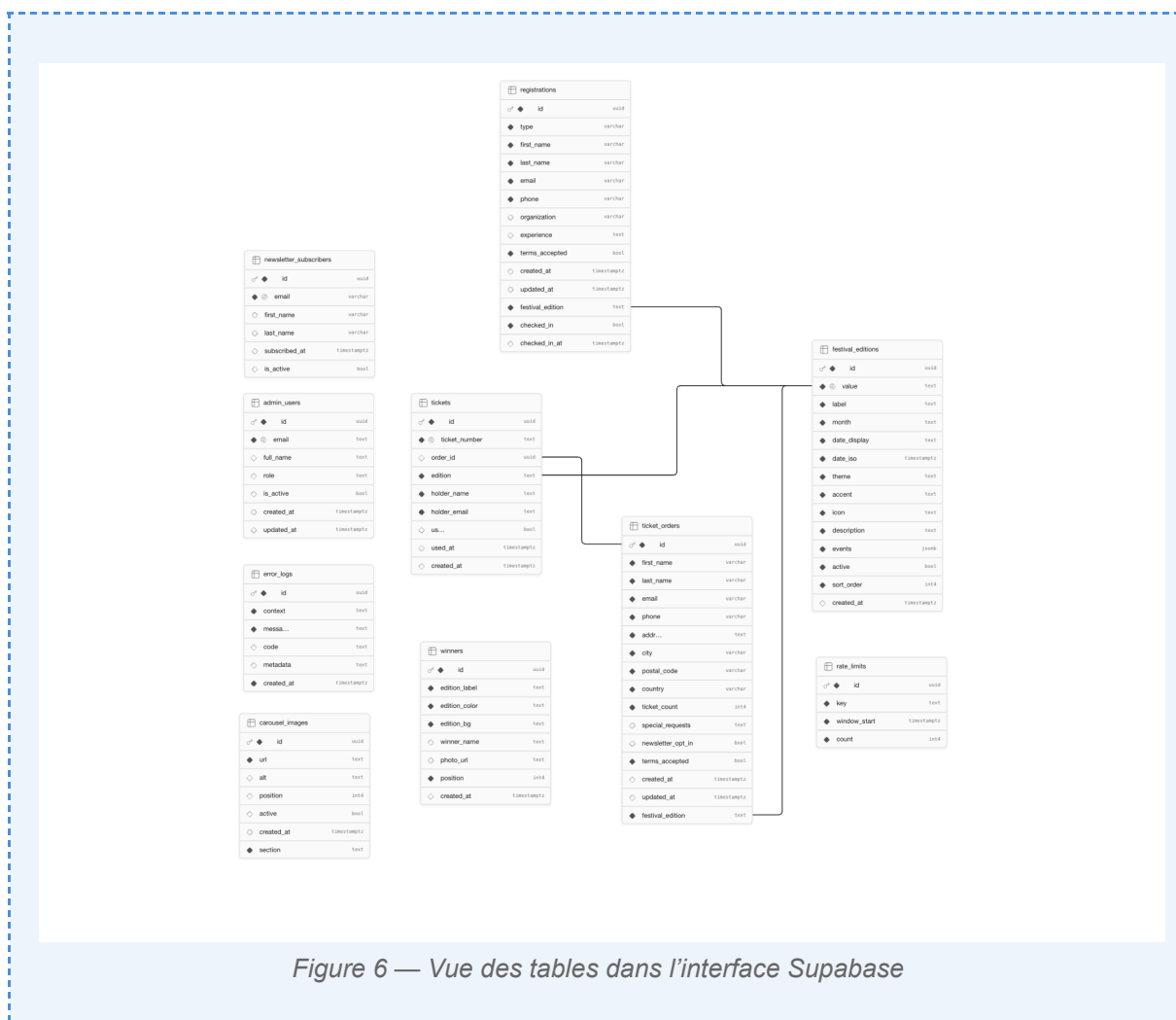


Figure 5 — Carte du marché — modal avec les 9 marqueurs gastronomiques

Chapitre 4 — Analyse de la base de données

4.1 Structure des tables

La base de données PostgreSQL hébergée sur Supabase est composée de dix tables dont voici la description et les relations (voir Figure 6).



La table `registrations` stocke les inscriptions au concours. Elle contient les informations personnelles du participant (prénom, nom, e-mail, téléphone, organisation éventuelle), sa catégorie (amateur ou pro via une contrainte CHECK), l'édition choisie via une clé étrangère vers `festival_editions`, le champ d'expérience et l'acceptation des conditions. Les champs `checked_in` et `checked_in_at` permettent de valider la présence du participant à l'entrée.

La table `ticket_orders` centralise les commandes de billets. Elle enregistre les coordonnées complètes du visiteur, le nombre de billets commandés (limité entre 1 et 10 par contrainte SQL CHECK), l'édition concernée et l'opt-in newsletter.

La table `tickets` représente les billets individuels générés. Chaque billet porte un numéro unique au format SWF-XXX-XXXXXX, est lié à `ticket_orders` (via `order_id`) et à `festival_editions` (via `edition`), et peut être marqué comme utilisé via les champs `used` et `used_at`.

La table `festival_editions` gère les éditions de façon entièrement dynamique, avec ses métadonnées complètes (identifiant `value`, libellé, date ISO, thème, couleur, icône Material Symbols, liste d'événements en JSONB). Les champs `sort_order` et `active` permettent de contrôler l'ordre et la visibilité sans suppression.

La table `admin_users` gère les comptes d'administration indépendamment de `auth.users`, avec un rôle (`admin`, `super_admin` ou `scanner` via contrainte CHECK) et un indicateur `is_active`. Les tables `carousel_images`, `newsletter_subscribers`, `winners`, `rate_limits` et `error_logs` complètent le schéma.

4.2 Relations entre les tables

La relation principale est celle entre `ticket_orders` et `tickets` (one-to-many). `registrations` et `ticket_orders` entretiennent chacune une relation many-to-one avec `festival_editions` via une clé étrangère sur le champ `value`. Les composants `Registration.jsx`, `Tickets.jsx` et `ScheduleSection.jsx` consomment les éditions via le hook `useEditions`. Les tables `admin_users`, `carousel_images`, `winners`, `rate_limits` et `error_logs` sont indépendantes et communiquent via les politiques RLS.

Chapitre 5 — Fonctionnalités clés du site

5.1 Le compte à rebours dynamique (Hero)

La section Hero affiche un compte à rebours en temps réel jusqu'à la prochaine édition. La logique repose sur un hook personnalisé `useCountdown` qui calcule la différence entre la date cible et l'instant présent via `setInterval`, et sur `useEditions` qui charge les éditions dynamiquement — `getNextEdition` retourne automatiquement la prochaine date future, sans rien coder en dur.

L'ambiance hivernale se joue dans les détails : des animations `animate-ping` créent un effet de scintillement d'étoiles autour du logo. Sur mobile, un clic sur le flocon déclenche une chute de neige interactive — 15 flocons générés avec des paramètres aléatoires (position, taille, durée, délai), positionnés en `fixed` avec `pointerEvents: none` pour ne perturber aucune interaction, qui disparaissent automatiquement après quelques secondes.

5.2 Gestion dynamique des éditions

L'`EditionsManager` permet à l'administrateur de créer, modifier, activer/désactiver et supprimer des éditions sans aucun redéploiement. Les composants `Registration.jsx`, `Tickets.jsx` et `ScheduleSection.jsx` consomment tous les éditions via `useEditions` — toute modification dans le back-office est immédiatement reflétée dans l'interface publique (voir Figure 7).

Snow Wonder ADMIN-IN-THE-SNOW ADMIN DÉCONNEXION

TABLEAU DE BORD INSCRIPTIONS BILLETTERIE SCANNER ÉDITIONS GALERIE GAGNANTS

Éditions du festival

Gérez les éditions affichées sur le site et dans les formulaires [+ Nouvelle édition](#)

Édition	Statut	Désactiver	Modifier	Supprimer
Édition d'Ouverture 6 décembre 2026 · Édition Ouverture	Actif	Désactiver	Modifier	Supprimer
Édition Mi-Hiver 10 janvier 2027 · Édition Mi-Hiver	Actif	Désactiver	Modifier	Supprimer
Grande Finale 7 février 2027 · Grande Finale	Actif	Désactiver	Modifier	Supprimer
test 31 main 2026 · Édition Spéciale	Inactif	Activer	Modifier	Supprimer
Quam qui deleniti qu In occaecat quis mod · Grande Finale	Inactif	Activer	Modifier	Supprimer

Nouvelle édition

IDENTIFIANT * LABEL * MOIS

ex: december_2026 Édition Décembre Décembre

Clé unique, non modifiable après création

DATE AFFICHÉE DATE ISO * ORDRE

6 décembre 2026 jj/mm/aaaa --:-- 0

THÈME APERÇU

— Choisir un thème — #4289b6 - ac_unit

DESCRIPTION

ÉVÉNEMENTS DU PROGRAMME

Trophée amateur Description de l'événement

+ Ajouter un événement

☒ Édition active (visible sur le site et dans les formulaires)

Annuler Enregistrer

Figure 7 — Onglet Éditions du back-office — formulaire de création/modification d'une édition

5.3 Les formulaires : inscription et billetterie

Deux formulaires distincts, présentés côte à côte, tous deux alimentés dynamiquement via `useEditions`.

Le formulaire d'inscription (`Registration.jsx`) permet aux participants de choisir leur catégorie (amateur ou professionnel) et leur(s) édition(s). Dès la saisie de l'e-mail, un mécanisme de détection en temps réel vérifie les éditions déjà inscrites et les désactive dans la liste — seules les nouvelles inscriptions sont insérées à la soumission. Une Edge Function envoie ensuite un e-mail de confirmation par édition.

Le formulaire de billetterie ([Tickets.jsx](#)) permet de réserver jusqu'à 10 billets gratuits par adresse e-mail et par édition. Chaque billet PDF est généré côté serveur avec [pdf-lib](#) : QR code unique, nom du porteur, référence de commande (voir Figure 8).

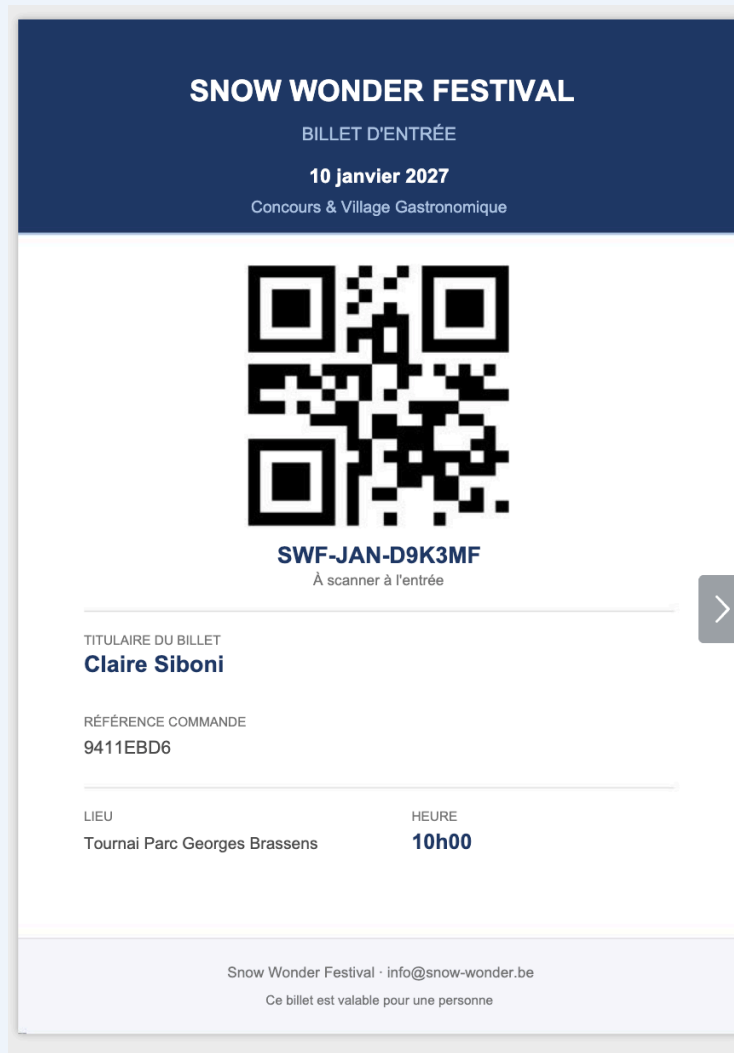


Figure 8 — Exemple de billet PDF généré avec QR code

5.4 La galerie photo dynamique

[GallerySection.jsx](#) charge ses images directement depuis [carousel_images](#) de Supabase, en filtrant les images actives ordonnées par position. Le lightbox est implémenté manuellement : navigation clavier (flèches, Escape) et compteur de position.

5.5 Le système de scan à l'entrée (TicketScanner)

Le système de validation repose sur `html5-qrcode` pour accéder à la caméra et décoder les QR codes en temps réel. Le retour visuel est immédiat : vert (validé), ambre (déjà scanné), rouge (introuvable) (voir Figure 9). Une saisie manuelle reste disponible en alternative. Le rôle `scanner` dans `admin_users` donne accès à cette interface épurée sans exposer le reste du back-office.

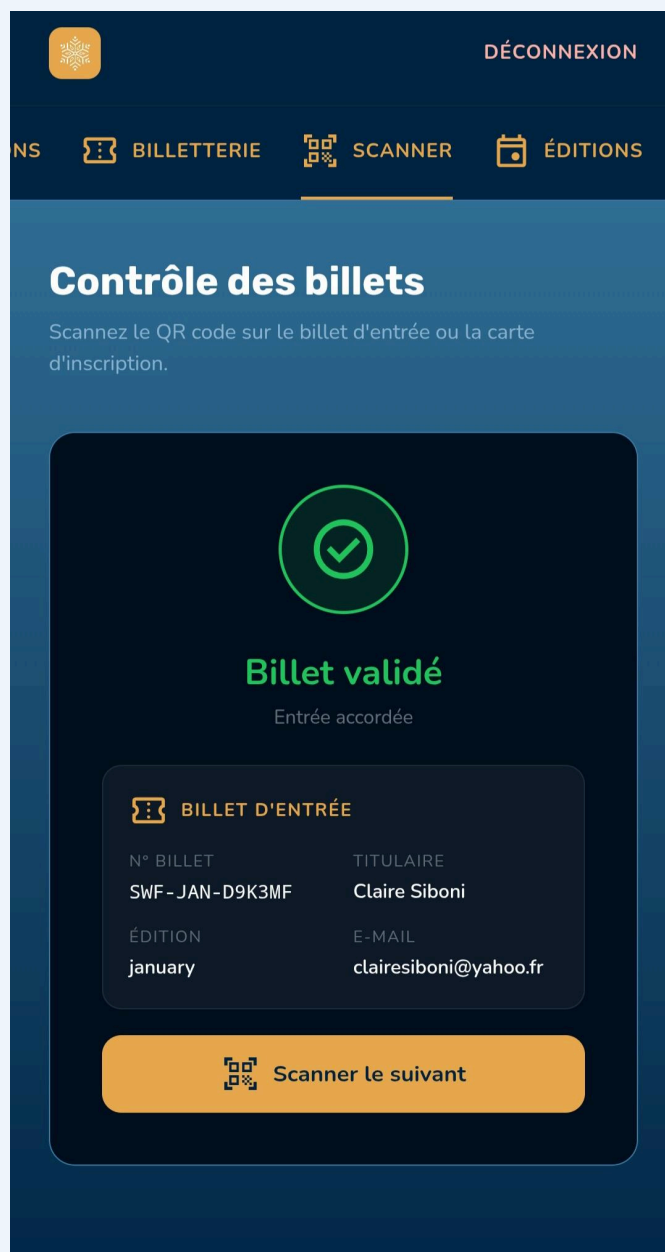
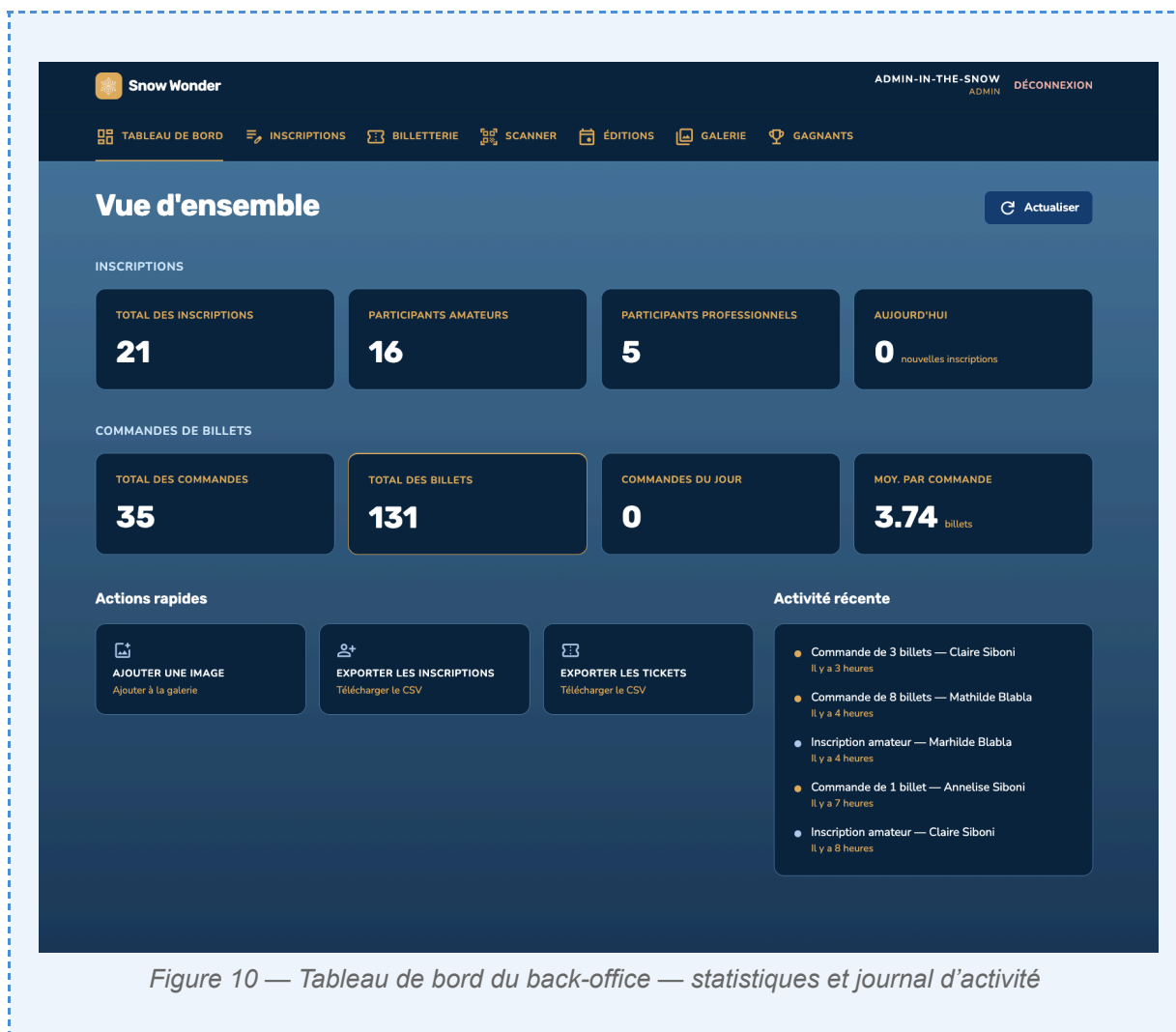


Figure 9 — Interface TicketScanner — retour visuel vert / ambre / rouge

5.6 Le panneau d'administration (Back-Office)

L'interface d'administration est accessible via `/#admin`. L'authentification passe par le système JWT de Supabase avec double vérification (Supabase Auth + table `admin_users`) — le rôle détermine ce qui est affiché : panneau complet pour `admin` et `super_admin`, interface de scan seule pour `scanner`.

Le panneau est structuré en sept onglets : Tableau de bord (statistiques, actions rapides, journal d'activité), Inscriptions, Billetterie, Scanner, Éditions, Galerie et Gagnants (voir Figures 10 et 11).





5.7 Sécurité et protection des données

La sécurité de l'application repose sur quatre couches complémentaires, pensées ensemble plutôt qu'ajoutées après coup.

Jamais de secret côté client. La clé anonyme Supabase ([ANON_KEY](#)) est publique par conception — elle ne permet que les opérations explicitement autorisées par les politiques RLS. Les secrets sensibles (clé SMTP, clé service_role, token Mapbox) ne vivent que dans les variables d'environnement des Edge Functions, côté serveur. Ils ne transitent jamais par le navigateur.

Le RLS comme unique garde-fou côté base de données. C'est la seule couche qui empêche un visiteur de lire les données des autres. Elle est configurée directement au niveau PostgreSQL, indépendamment du code applicatif — ce qui signifie qu'un bug dans le front-end ne peut pas la contourner.

Validation et déduplication à chaque niveau. Les données sont validées côté client avant soumission, puis contraintes au niveau SQL (types, CHECK, unicité). La fonction send-contact applique un échappement HTML explicite sur toutes les variables avant insertion dans le template. React échappe automatiquement les valeurs interpolées dans le JSX, ce qui couvre la protection XSS côté rendu.

Rate limiting sur les Edge Functions d'envoi d'e-mails. Pour éviter tout abus, chaque fonction est limitée par adresse e-mail et par heure :

Fonction	Limite
send-confirmation (inscription concours)	5 envois par e-mail par heure
send-ticket (billetterie)	5 envois par e-mail par heure
send-contact (formulaire de contact)	3 messages par e-mail par heure

Lorsque la limite est atteinte, la fonction retourne un code HTTP 429 Too Many Requests.

Flux splitté pour la billetterie. Le client insère la commande directement dans `ticket_orders`, puis délègue la logique sensible à l'Edge Function `send-ticket` — qui tourne avec les droits `service_role` — pour insérer les billets, générer les PDF et envoyer les e-mails. Le navigateur ne touche jamais aux opérations qui nécessitent des privilèges élevés. (voir Figure 12)

```

Deno.serve(async (req) => {
  if (req.method === "OPTIONS") {
    return new Response("ok", { headers: corsHeaders });
  }

  try {
    const {
      name: nom,
      email,
      edition,
      editionLabel = edition,
      editionDate = "",
      editionTime = "",
      quantity: quantite,
      orderId: idCommande,
    } = await req.json();
    console.log(
      `send-confirmation-billets: ${email} edition=${edition} qte=${quantite}`,
    );

    // 5 envois max par email par heure (protection anti-spam)
    const allowed = await checkRateLimit(
      `ticket:${email.toLowerCase()}`,
      5,
      3600,
    );
    if (!allowed) {
      return new Response(JSON.stringify({ error: "Trop de requêtes." }), {
        status: 429,
        headers: { ...corsHeaders, "Content-Type": "application/json" },
      );
    }

    const billets = Array.from({ length: quantite }, () => ({
      ticket_number: genererNumeroBillet(edition),
      order_id: idCommande,
      edition,
      holder_name: nom,
      holder_email: email,
    }));

    const dbRes = await fetch(`${SUPABASE_URL}/rest/v1/tickets`, {
      method: "POST",
      headers: {
        Authorization: `Bearer ${SUPABASE_SERVICE_KEY}`,
        apikey: SUPABASE_SERVICE_KEY,
        "Content-Type": "application/json",
        Prefer: "return=minimal",
      },
      body: JSON.stringify(billets),
    });

    if (!dbRes.ok) {
      const err = await dbRes.text();
      throw new Error(`Échec insertion DB : ${err}`);
    }

    let piecesJointes: object[] = [];
    try {
      piecesJointes = await Promise.all(
        billets.map(async (b) => {
          const pdfBytes = await genererBilletPDF(
            b.ticket_number,
            nom,
            edition,
            idCommande,
            editionLabel,
            editionDate,
            editionTime,
          );
          return {
            filename: `billet-${b.ticket_number}.pdf`,
            content: Buffer.from(pdfBytes),
            contentType: "application/pdf",
          };
        }),
      );
      console.log(`${piecesJointes.length} PDF(s) prêt(s)`);
    } catch (erreurPdf) {
      console.error("Échec PDF :", erreurPdf);
    }

    const html = emailConfirmationBillets({
      nom,
      email,
      edition,
      editionLabel,
      editionDate,
      editionTime,
      quantite,
      idCommande,
    });

    const transporteur = nodemailer.createTransport({
      host: SMTP_HOST,
      port: SMTP_PORT,
      secure: SMTP_PORT === 465,
      auth: { user: SMTP_USER, pass: SMTP_PASS },
    });

    await transporteur.sendMail({
      from: FROM,
      to: email,
      subject: `Vos billets - Snow Wonder Festival ${edition}`,
      html,
      attachments: piecesJointes,
    });

    console.log("Email envoyé");
  }
});

```

Figure 12 — Extrait de code — Edge Function `send-ticket` (génération PDF + envoi e-mail)

Chapitre 6 — Déploiement et mise en production

6.1 Déploiement sur Vercel

Le site est déployé sur Vercel, qui détecte automatiquement le projet Vite, lance `npm run build` et sert le dossier `dist/` généré. Vercel Analytics a été intégré pour suivre les métriques de trafic de manière respectueuse de la vie privée (voir Figure 13).

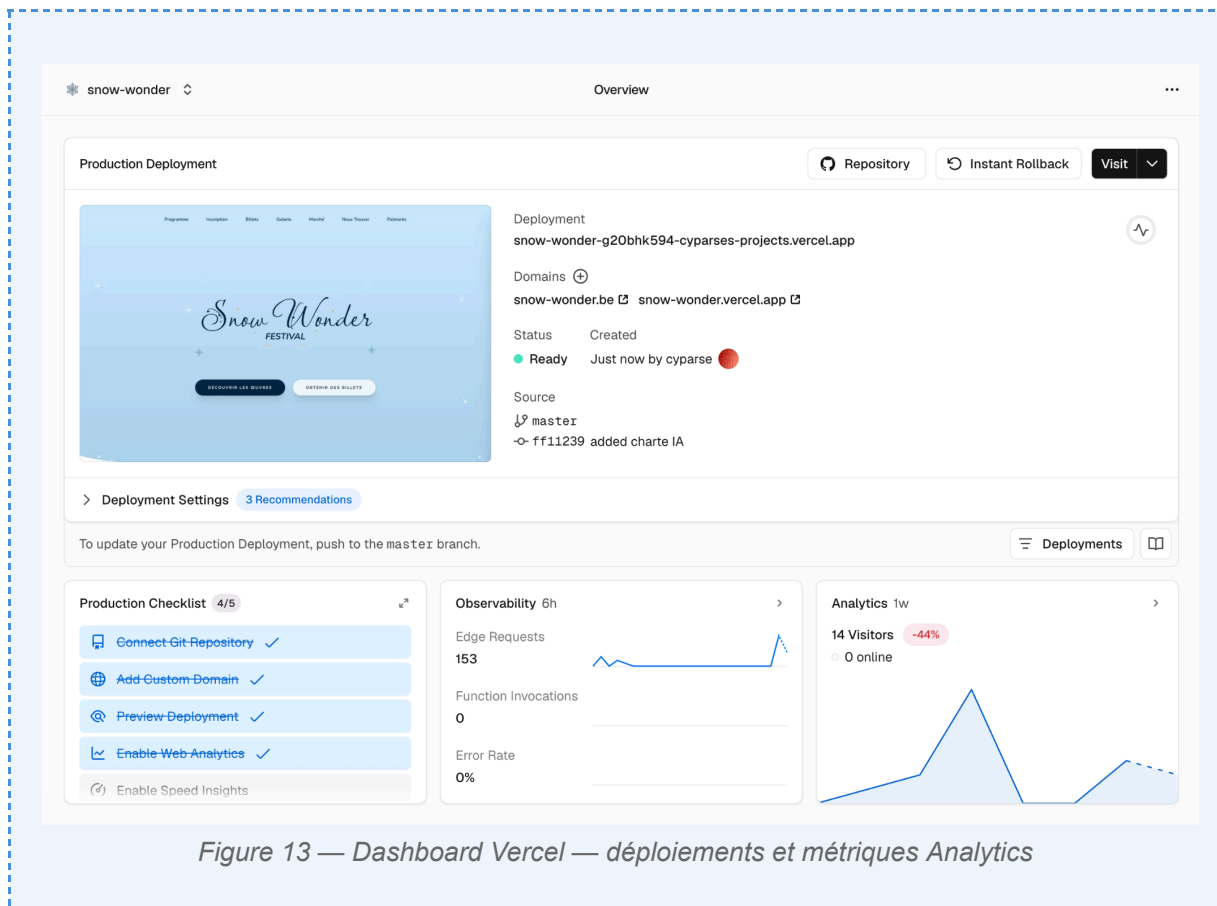


Figure 13 — Dashboard Vercel — déploiements et métriques Analytics

6.2 Supabase Edge Functions

Les Edge Functions Deno (`send-confirmation`, `send-contact`, `send-ticket`) sont déployées sur l'infrastructure Supabase, exécutées à la périphérie du réseau. La gestion des en-têtes CORS est mutualisée dans un fichier partagé `_shared/cors.ts`.

6.3 Optimisation des performances

Un audit PageSpeed sur le site déployé a mis en évidence quatre problèmes : un bundle JavaScript monolithique de 296 Ko, des images sans attributs `width/height` générant du CLS, des ressources graphiques trop lourdes, et six tâches longues sur le thread principal.

Découpage du bundle. Grâce à `React.lazy` et `<Suspense>`, les sections sous la ligne de flottaison et la totalité du back-office sont chargées à la demande. La configuration Vite définit des chunks stables — `vendor-react`,

`vendor-supabase`, `vendor-leaflet` et `admin`. Le chunk initial est descendu de 296 Ko à environ 220 Ko.

Lazy loading et priorisation des images. Les attributs `loading="lazy"` et `decoding="async"` ont été appliqués à toutes les images hors zone visible. L'image LCP (Hero) reçoit `fetchpriority="high"` pour un téléchargement prioritaire.

Conversion WebP. Les trois images locales les plus lourdes ont été recompressées à 80 % — le fond du marché passe de 8,1 Mo à 231 Ko, le footer de 2,4 Mo à 111 Ko, l'image héro de 1,4 Mo à 388 Ko. Les images de la galerie sur Supabase Storage ont également été converties en WebP avec une largeur maximale de 1 200 px. La vigilance sur le poids des images reste un effort continu : c'est le type de dette qui s'accumule silencieusement dès qu'on commence à alimenter une galerie.

Chapitre 7 — RGPD et cookies

7.1 RGPD et protection des données personnelles

Le Règlement Général sur la Protection des Données (RGPD) encadre la collecte, le stockage et le traitement de toute donnée personnelle au sein de l'Union européenne. Dans le cadre de Snow Wonder Festival, deux catégories de données personnelles sont collectées selon le formulaire utilisé.

Pour l'inscription au concours : prénom, nom, adresse e-mail, numéro de téléphone et organisation éventuelle.

Pour la réservation de billets : prénom, nom, adresse e-mail, numéro de téléphone et adresse postale complète (rue, ville, code postal, pays).

Les mesures de conformité mises en œuvre sont les suivantes. Les formulaires soumettent les données exclusivement via HTTPS chiffré. Les politiques RLS de PostgreSQL garantissent que les visiteurs anonymes ne peuvent qu'insérer des données. Les secrets sont stockés dans les variables d'environnement Supabase. Les utilisateurs acceptent explicitement les conditions via une case à cocher obligatoire avant toute soumission, conformément au principe de consentement éclairé du RGPD. L'administrateur peut supprimer toute inscription ou commande pour honorer les demandes de droit à l'effacement.

7.2 Gestion des cookies

L'application utilise des cookies techniques Supabase pour la gestion des sessions JWT de l'administrateur, strictement nécessaires au fonctionnement du back-office. Vercel Analytics collecte des métriques agrégées et anonymisées, sans cookies de traçage tiers.

Chapitre 8 — Estimation budgétaire

8.1 Hypothèses de calcul

L'estimation repose sur le tarif horaire moyen d'un développeur web junior en Belgique, soit 29,24 €/heure brut selon Joooble¹. Le projet étant réalisé en totale autonomie, aucun coût de coordination d'équipe n'est à prévoir.

8.2 Phases de développement

Phase	Description	Durée	Coût
Analyse	Brief client, impact mapping, user stories, planification	10h	292,40 €
Conception	Charte graphique, wireframes, schéma de base de données	20h	584,80 €
Développement front-end	Composants React, TailwindCSS, intégration Leaflet/Mapbox	60h	1 754,40 €
Développement back-end	Configuration Supabase, RLS, Edge Functions, back-office	50h	1 462,00 €
Tests et débogage	Tests fonctionnels multi-navigateurs, corrections	15h	438,60 €
Déploiement	Mise en ligne Vercel, migration base de données, domaine	10h	292,40 €
Total développement		165h	4 824,60 €

8.3 Frais annexes (première année)

Poste	Coût annuel estimé
Nom de domaine snow-wonder.be (OVH)	~10,14 €
Adresse e-mail professionnelle (OVH, 0,30 €/mois)	~3,60 €
Hébergement Vercel (plan gratuit)	0 €
Supabase (plan gratuit, jusqu'à 50 000 utilisateurs actifs mensuels)	0 €
Total frais techniques	~13,74 €

8.4 Maintenance annuelle

La maintenance couvre les mises à jour des dépendances (React, TailwindCSS v4, Supabase SDK), la surveillance des performances, les corrections de bugs éventuels

¹JOOBLE, joooble.org, <https://be.joooble.org/salary/juniordeveloper>, 21/05/2026, 11/12/2024 — « La statistique du tarif horaire moyen d'un développeur junior en Belgique m'a permis d'établir les critères de rémunération »

et l'ajout de contenu dans le back-office. Elle est estimée à 10 heures par an, soit 292,40 €.

8.5 Coût total première année

Poste	Montant
Développement	4 824,60 €
Frais techniques	~13,74 €
Maintenance	292,40 €
Total	~5 130,74 €

Ce budget constitue une estimation indicative pour un développeur junior travaillant en autonomie.

Conclusion

Intégration des savoirs et savoir-faire

Ce projet a été l'occasion de mettre en pratique, souvent pour la première fois en conditions réelles, des compétences que la formation avait introduites sans toujours pouvoir les approfondir. Configurer des politiques RLS directement au niveau PostgreSQL, orchestrer des Edge Functions Deno pour l'envoi d'e-mails transactionnels et la génération de PDF, intégrer proprement une bibliothèque DOM impérative comme Leaflet dans un environnement React — chacun de ces défis m'a obligée à comprendre les mécanismes sous-jacents plutôt qu'à me contenter d'une solution qui fonctionne sans savoir pourquoi.

Le volet design a été tout aussi formateur. Concevoir trois expériences visuelles distinctes pour mobile, tablette et desktop — sans masquer des éléments, mais en repensant réellement la mise en page à chaque breakpoint — m'a appris à raisonner en termes de contexte d'utilisation autant qu'en termes de technique. Ce n'est pas la même chose de cacher un composant et de décider qu'il n'a pas sa place sur ce format.

Adéquation entre objectifs et résultats

Le brief demandait un site événementiel. Ce qui a été livré, c'est une application complète : front-office public, back-office sécurisé, carte interactive avec marqueurs personnalisés, billetterie avec génération de PDF et envoi automatique, gestion des éditions sans redéploiement, et un système de scan à l'entrée avec rôle dédié. Chaque objectif du cahier des charges a été atteint — et plusieurs n'y figuraient pas.

Difficultés rencontrées et solutions apportées

La refonte technologique a été le premier obstacle sérieux. Passer de HTML/CSS/JS vanilla à React et TailwindCSS en cours de projet, c'est reprendre une base existante et la restructurer entièrement sans perdre le fil des fonctionnalités en cours. C'était inconfortable. C'était aussi inévitable — et rétrospectivement, c'est la partie du projet dont je suis le plus satisfaite techniquement.

La section formulaires sur desktop a exigé de nombreuses itérations : trouver l'équilibre entre l'effet visuel des deux cercles translucides superposés et l'utilisabilité réelle des formulaires qu'ils contiennent n'avait rien d'évident. Et la gestion dynamique des éditions a révélé un bug RLS silencieux — l'inscription à la newsletter échouait lors de la création d'une nouvelle édition, sans message d'erreur explicite. Ce genre de bug, qu'on ne trouve qu'en production, enseigne plus que n'importe quel exercice de formation.

Réflexion personnelle

Snow Wonder Festival est le projet qui m'a le plus appris — non pas parce qu'il était guidé, mais précisément parce qu'il ne l'était pas. Personne pour valider les choix d'architecture, personne pour débloquer un bug à 23h, personne pour dire si la

décision de tout reprendre en React était raisonnable ou non. Juste le projet, les docs, et l'obligation de trouver.

Je repars de ce TFE avec une conviction : c'est dans cet environnement-là — exigeant, rarement confortable, en constante évolution — que je veux continuer à travailler. Pas malgré la complexité. À cause d'elle.

Bibliographie

Ouvrages et règlements

- ❖ COMMISSION EUROPÉENNE, europa.eu, Règlement (UE) 2016/679 — RGPD, <https://eur-lex.europa.eu/legal-content/FR/TXT/?uri=CELEX%3A32016R0679>, 21/05/2026, 27/04/2016
- ❖ ADEME, ademe.fr, La face cachée du numérique, <https://www.ademe.fr/la-face-cachee-du-numerique>, 21/05/2026, 2019

Documentations techniques

- ❖ REACT, react.dev, Documentation officielle React 19, <https://react.dev>, 21/05/2026
- ❖ VITEJS, vitejs.dev, Documentation officielle Vite, <https://vitejs.dev>, 21/05/2026
- ❖ TAILWINDCSS, tailwindcss.com, Documentation officielle TailwindCSS v4, <https://tailwindcss.com>, 21/05/2026
- ❖ SUPABASE, supabase.com, Documentation officielle — Row Level Security, <https://supabase.com/docs/guides/auth/row-level-security>, 21/05/2026
- ❖ LEAFLET, leafletjs.com, Documentation officielle Leaflet.js, <https://leafletjs.com>, 21/05/2026
- ❖ MAPBOX, mapbox.com, Documentation Mapbox Styles API, <https://docs.mapbox.com/api/maps/styles/>, 21/05/2026
- ❖ VERCEL, vercel.com, Documentation officielle Vercel, <https://vercel.com/docs>, 21/05/2026
- ❖ HTML5-QRCODE, github.com/mebjas/html5-qrcode, Documentation officielle html5-qrcode, <https://github.com/mebjas/html5-qrcode>, 21/05/2026

Sources économiques

- ❖ JOOBLE, jooble.org, Salaire développeur junior Belgique, <https://be.jooble.org/salary/juniordeveloper>, 21/05/2026, 11/12/2024

Crédits photographiques

- ❖ NESTER Siarhei, pexels.com, Photo neige/hiver, <https://www.pexels.com/fr-fr/photo/neige-eau-hiver-glace-18858820/>, 21/05/2026
- ❖ SIDDIQUE Saif, pexels.com, Sculpture de neige grenouille guitariste, <https://www.pexels.com/fr-fr/photo/sculpture-de-neige-d-une-grenouille-jouant-de-la-guitare-a-frankenmuth-35252683/>, 21/05/2026
- ❖ GROSSGASTEIGER Eberhard, pexels.com, Lampe château hivernal, <https://www.pexels.com/fr-fr/photo/lampe-a-theme-chateau-beige-776389/>, 21/05/2026
- ❖ RADIK 2707, pexels.com, Sculpture de glace nocturne, <https://www.pexels.com/fr-fr/photo/creation-d-une-sculpture-de-glace-nocturne-en-noir-et-blanc-30238611/>, 21/05/2026

- ❖ SOMMACAL Francesco, pexels.com, Enfant et bonhomme de neige, <https://www.pexels.com/fr-fr/photo/enfant-construisant-un-grand-bonhomme-de-neige-dans-un-paysage-hivernal-36242337/>, 21/05/2026
- ❖ BOLTNEVA Valeria, pexels.com, Hiver froid, <https://www.pexels.com/fr-fr/photo/rhume-froid-neige-hiver-14817812/>, 21/05/2026
- ❖ S. Natalia, pexels.com, Famille bonhommes de neige, <https://www.pexels.com/fr-fr/photo/charmante-famille-de-bonhommes-de-neige-lors-d-une-journee-d-hiver-30129593/>, 21/05/2026
- ❖ YUAN Yuan, pexels.com, Bonhomme de neige décoré, <https://www.pexels.com/fr-fr/photo/joyeux-bonhomme-de-neige-decore-de-feuilles-d-automne-29567493/>, 21/05/2026
- ❖ AUTEUR INCONNU, pxhere.com, Photo hivernale, <https://pxhere.com/en/photo/1072102>, 21/05/2026
- ❖ AUTEUR INCONNU, pxhere.com, Photo hivernale, <https://pxhere.com/en/photo/1184648>, 21/05/2026
- ❖ Rincewind42, wikimedia.org, Harbin Ice Festival sculpture, https://upload.wikimedia.org/wikipedia/commons/a/a4/Moomen_in_Snow%2C_Harbin_International_Ice_and_Snow_Sculpture_Festival_%283238489838%29.jpg, 21/05/2026
- ❖ HARKNESS Derek, flickr.com, Harbin festival photo, <https://www.flickr.com/photos/derekharkness/3237690883>, 21/05/2026
- ❖ Rincewind42, wikimedia.org, Snow steam train Harbin, https://upload.wikimedia.org/wikipedia/commons/9/94/Snow_steam_train%2C_Harbin_International_Ice_and_Snow_Sculpture_Festival_%283237649999%29.jpg, 21/05/2026

Intelligence artificielle

ANTHROPIC, claude.ai, Claude Sonnet 4.6, <https://claude.ai>, mai 2026, août 2025

- ❖ Reformulation et amélioration rédactionnelle de certains passages du texte
- ❖ Débogage (rendu email/PDF des tickets, gestion des erreurs, carrousel, section admin mobile) — mars–mai 2026
- ❖ Correction des politiques RLS Supabase — avril 2026
- ❖ Vérification de sécurité finale — mai 2026
- ❖ Mise en place d'une librairie QR et génération PDF — mai 2026
- ❖ Génération d'image pour la section marketplace — mars 2026

Table des figures

- Figure 1 — Page d'accueil — section Hero avec compte à rebours (desktop)
- Figure 2 — Section formulaires — mise en page desktop (cercles superposés)
- Figure 3 — Section formulaires — mise en page mobile (quart de cercle)
- Figure 4 — Carte principale Leaflet — marqueur du site du festival
- Figure 5 — Carte du marché — modal avec les 9 marqueurs gastronomiques
- Figure 6 — Vue des tables dans l'interface Supabase (Table Editor ou diagramme ERD)
- Figure 7 — Onglet Éditions du back-office — formulaire de création/modification d'une édition
- Figure 8 — Exemple de billet PDF généré avec QR code
- Figure 9 — Interface TicketScanner — retour visuel vert / ambre / rouge
- Figure 10 — Tableau de bord du back-office — statistiques et journal d'activité
- Figure 11 — Onglet Billetterie — liste des commandes et billets générés
- Figure 12 — Extrait de code — Edge Function send-ticket (génération PDF + envoi e-mail)
- Figure 13 — Dashboard Vercel — déploiements et métriques Analytics

Annexes

Annexe A — Schéma du chemin des données

Ce chapitre décrit l'intégralité des flux de données de l'application, depuis le navigateur du visiteur jusqu'aux services tiers, en passant par Supabase et les Edge Functions.

A.1 Vue d'ensemble de l'architecture

L'application s'articule autour de trois pôles : le navigateur client (React 19 + Vite, hébergé sur Vercel), Supabase (PostgreSQL avec RLS, Auth JWT, Edge Functions Deno/TypeScript et Storage), et les services tiers (api.mapbox.com pour les tuiles, api.qrserver.com pour les QR codes, Vercel Analytics et Google Maps). Les envois d'e-mails transitent par un serveur SMTP via Nodemailer, appelé depuis les Edge Functions.

A.2 Parcours 1 — Chargement dynamique des éditions

Au montage des composants Registration.jsx, Tickets.jsx et ScheduleSection.jsx, le hook personnalisé useEditions interroge la table festival_editions de Supabase pour récupérer la liste des éditions actives. Les éditions ne sont plus définies en dur dans le code source : toute modification effectuée dans l'EditionsManager du back-office est immédiatement reflétée dans l'interface publique.

A.3 Parcours 2 — Inscription au concours

Le visiteur saisit son e-mail (délai de 500 ms) — le composant Registration.jsx interroge la table registrations pour détecter les éditions déjà inscrites et les affiche en temps réel. À la soumission, les données sont insérées dans registrations (RLS anon INSERT autorisé), puis l'Edge Function send-confirmation génère et envoie un e-mail HTML personnalisé.

A.4 Parcours 3 — Réservation de billets

À la soumission : (1) insertion dans ticket_orders (RLS anon INSERT) ; (2) appel à l'Edge Function send-ticket qui génère N numéros au format SWF-{EDI}-{6 chars}, insère les billets dans tickets (service_role), génère les PDF via pdf-lib avec QR codes et envoie les billets en pièces jointes via SMTP ; (3) si newsletter_opt_in = true, upsert dans newsletter_subscribers.

A.5 Parcours 4 — Formulaire de contact

La soumission de la modal ContactModal.jsx invoque l'Edge Function send-contact, qui transmet le message par e-mail à info@snow-wonder.be. Aucune insertion en base de données n'est effectuée.

A.6 Parcours 5 — Galerie photo (lecture publique)

Au montage de GallerySection.jsx, une requête SELECT filtre les images actives (active = true, section = 'gallery') ordonnées par position. Un lightbox implémenté manuellement gère la navigation clavier.

A.7 Parcours 6 — Scan à l'entrée (TicketScanner)

L'agent de contrôle accède à l'interface via le rôle scanner. Le composant TicketScanner.jsx utilise html5-qrcode pour accéder à la caméra et décoder les QR codes en temps réel. L'interface retourne un code couleur immédiat : vert (validation réussie), orange (déjà scanné), rouge (introuvable).

A.8 Parcours 7 — Authentication Back-Office

La connexion passe par supabase.auth.signInWithPassword(), suivie d'une double vérification dans admin_users. Le rôle détermine l'interface affichée : panneau complet pour admin/super_admin, interface de scan pour scanner.

A.9 Parcours 8 — Back-Office (opérations CRUD admin)

Onglet	Opérations
Dashboard	SELECT COUNT registrations, ticket_orders — actions rapides — journal d'activité récente
Éditions	SELECT / INSERT / UPDATE / DELETE FROM festival_editions — aperçu icône Material Symbols
Inscriptions	SELECT / UPDATE / DELETE FROM registrations
Billetterie	SELECT / UPDATE / DELETE FROM ticket_orders, tickets
Galerie	SELECT / INSERT / UPDATE / DELETE FROM carousel_images
Gagnants	SELECT / INSERT / UPDATE / DELETE FROM winners
Scanner	Lecture/mise à jour de tickets (used/used_at) et registrations (checked_in/checked_in_at)

A.10 Tableau récapitulatif des permissions RLS

TABLE	INSERT	SELECT	UPDATE / DELETE
registrations	anon	authenticated	authenticated
ticket_orders	anon	authenticated	authenticated
tickets	service_role	authenticated + scanner	authenticated + scanner
festival_editions	authenticated	anon + auth	authenticated
newsletter_subscribers	anon	authenticated	authenticated
carousel_images	authenticated	anon + auth	authenticated
winners	authenticated	anon + auth	authenticated
rate_limits	service_role	service_role	service_role
admin_users	—	authenticated	—



Date d'édition : 21-01-2025

Usage de l'Intelligence Artificielle dans le cadre du Travail de Fin d'Etudes (TFE)

Le présent document complété, daté et signé est à intégrer en première page du TFE.

Cadre réservé à l'apprenant :

Dans le cadre de mon TFE (cocher la ou les usages) :

- ☐ J'ai exploité un outil IA pour m'aider à structurer mes idées
- ☒ J'ai exploité un outil IA pour m'aider à formuler mes idées (correcteur d'orthographe-reformulateur de phrases)
- ☒ J'ai exploité un outil IA pour m'aider à illustrer visuellement mes idées (images d'illustration)
- ☒ J'ai exploité un outil IA pour un autre usage (précisez ci-dessous) :
débogage
- ☐ Je n'ai pas exploité d'outil IA

Le Centre IFAPME Wallonie picarde, soutenu par l'IFAPME, mène une politique d'acculturation et d'appropriation responsable des outils numériques issus de l'Intelligence artificielle.

Par sa signature, les jurys interne et externe prennent connaissance des actions menées par le Centre et des usages éventuels de l'IA par l'apprenant dans la rédaction de son TFE.

Par sa signature, l'apprenant formule explicitement l'usage éventuel de l'IA pour la rédaction de son TFE en toute transparence.

La Charte de l'IFAPME concernant l'usage de l'IA est accessible en scannant ce QR Code :



Apprenant	Jury interne	Jury externe
Nom Prénom : Claire Siboni	Nom Prénom :	Nom Prénom :
Date : 23 / 05 / 2026	Date : __ / __ / __	Date : __ / __ / __
Signature : 	Signature :	Signature :